

8-Week Software + AI Engineering Roadmap

4 Weeks MERN + Java Full Stack → Deployment & Monitoring | 4 Weeks AI/LLM → RAG, Embeddings & Agentic AI

Outcome: Build production-style projects, use Git/GitHub professionally, deploy them, add monitoring, and finish with an AI agentic application.

Recommended intensity: 6–8 focused hours/day. Every day: 60–90 min concepts + 3–4 hrs coding + 1–2 hrs project + 30 min Git/GitHub + 15–30 min revision.

Important: This is an accelerated syllabus. “Everything” in 8 weeks is not realistic at expert depth; the target is interview-ready fundamentals plus the ability to build, deploy, debug and extend real projects.

Daily Git/GitHub rule

Create one repository per major project. Use feature branches, meaningful commits, README updates, .gitignore, environment variables, issues/tasks, pull-request style reviews, tags/releases and a clean final README with architecture, setup, API docs, screenshots and deployment links.

8-week completion map

Week	Focus
Week 1 — MERN Foundations	HTML/CSS/JS → Git → React → Node/Express → MongoDB
Week 2 — MERN Production	Auth → REST APIs → validation → testing → frontend integration → security
Week 3 — Java Full Stack	Java → OOP → Collections → SQL → JDBC → Spring Boot → REST
Week 4 — Java Production + DevOps	Spring Security → JPA → testing → Docker → CI/CD → cloud → monitoring
Week 5 — AI/LLM Foundations	Python → ML basics → tokens → transformers → prompting → APIs
Week 6 — Embeddings + RAG	Embeddings → vector DB → retrieval → chunking → reranking → evaluation
Week 7 — Agentic AI	Tools → function calling → workflows → LangGraph → MCP → memory → guardrails
Week 8 — AI Production	Agent project → observability → evaluation → Docker → deployment → portfolio

W1D1 — Web + Git Foundations

Learn: HTML5 semantic structure, forms, accessibility basics; CSS box model, flexbox, grid, responsive design; Git init/status/add/commit/log/branch/merge; GitHub repo, README, .gitignore.

Completion target: Build a responsive developer portfolio landing page. Push initial version.

Git/GitHub: git init → branch feature/ui → commits → push → README.

W1D2 — JavaScript Core

Learn: Variables, types, operators, conditionals, loops, functions, scope, arrays, objects, destructuring, spread/rest, modules.

Completion target: Build a CLI/browser expense tracker with CRUD in localStorage.

Git/GitHub: Feature branches; commit after each feature.

W1D3 — Modern JS + Async

Learn: DOM, events, callbacks, promises, async/await, fetch, JSON, error handling, map/filter/reduce, modules.

Completion target: Build a weather/search dashboard using a public API.

Git/GitHub: README API section; .env if needed.

W1D4 — React Fundamentals

Learn: Vite, JSX, components, props, state, events, conditional rendering, lists, forms, hooks.

Completion target: Convert the dashboard into React.

Git/GitHub: Open issue per component; commit logically.

W1D5 — React Intermediate

Learn: useEffect, custom hooks, React Router, Context, reusable components, loading/error states.

Completion target: Build multi-page task/product dashboard.

Git/GitHub: Branch: feature/router; feature/context.

W1D6 — Node + Express

Learn: Node runtime, npm, modules, Express, middleware, REST, HTTP status codes, controllers, error handling.

Completion target: Create REST API for tasks/products.

Git/GitHub: API README + sample requests.

W1D7 — MongoDB + MERN Mini Project

Learn: MongoDB documents, collections, CRUD, indexes; Mongoose schemas/models; connect React → Express → MongoDB.

Completion target: PROJECT 1: MERN Task/Product Manager MVP.

Git/GitHub: Tag v0.1; GitHub project board; weekly retrospective.

W2D8 — REST Architecture

Learn: Routes/controllers/services/models; pagination, filtering, sorting, search; API response conventions.

Completion target: Add search, filters and pagination to Project 1.

Git/GitHub: Refactor commits; document endpoints.

W2D9 — Authentication

Learn: Password hashing, JWT/access tokens, refresh concepts, cookies vs localStorage, protected routes, roles.

Completion target: Add register/login/logout + protected dashboard.

Git/GitHub: Security-focused commits; never commit secrets.

W2D10 — Validation + Security

Learn: Input validation, CORS, Helmet, rate limiting concepts, sanitization, secure headers, OWASP basics.

Completion target: Harden Project 1 API.

Git/GitHub: Add SECURITY.md and environment template.

W2D11 — React Production Patterns

Learn: Form libraries/concepts, error boundaries, state patterns, API service layer, reusable UI, optimistic updates.

Completion target: Upgrade UX and error handling.

Git/GitHub: Create release checklist.

W2D12 — Testing

Learn: Unit/integration concepts; Jest/Vitest; API tests; React component tests; test data.

Completion target: Write tests for auth, CRUD and critical UI.

Git/GitHub: CI-ready test command.

W2D13 — Docker + Deployment

Learn: Dockerfile, images, containers, Docker Compose; build frontend/backend; environment configuration.

Completion target: Containerize Project 1.

Git/GitHub: Add Docker instructions to README.

W2D14 — MERN Production Release

Learn: Deploy frontend/backend/database; logs; health endpoint; basic uptime/error monitoring concepts.

Completion target: PROJECT 1 FINAL: deployed MERN app.

Git/GitHub: Tag v1.0; GitHub release; architecture diagram.

W3D15 — Java Core

Learn: JDK/JVM/JRE, syntax, types, operators, control flow, methods, packages, exceptions; IDE and Maven/Gradle concepts.

Completion target: Build a console banking/library app.

Git/GitHub: Maven project + README.

W3D16 — OOP Deep Dive

Learn: Classes/objects, constructors, encapsulation, inheritance, polymorphism, abstraction, interfaces; SOLID intro.

Completion target: Refactor app using layered OOP design.

Git/GitHub: Commit by domain feature.

W3D17 — Collections + Modern Java

Learn: List/Set/Map, generics, equals/hashCode, Comparator, streams, lambdas, Optional, records concepts.

Completion target: Implement search/reporting with streams.

Git/GitHub: Add tests for core logic.

W3D18 — SQL + DBMS for Java

Learn: Relational design, keys, joins, indexes, transactions, normalization; SELECT/INSERT/UPDATE/DELETE; MySQL.

Completion target: Design schema for an e-commerce/order system.

Git/GitHub: schema.sql + seed.sql in GitHub.

W3D19 — JDBC

Learn: Connection, PreparedStatement, ResultSet, CRUD, transactions, DAO pattern, connection pooling concept.

Completion target: Build Java CRUD API/console layer over MySQL.

Git/GitHub: DAO/service separation.

W3D20 — Spring Boot Basics

Learn: Spring Boot, Maven, dependency injection, configuration, profiles, REST controllers, services, repositories.

Completion target: Create Spring Boot REST API for products/orders.

Git/GitHub: API documentation + health endpoint.

W3D21 — Spring Data JPA

Learn: Entities, relationships, repositories, JPQL, pagination, transactions, DTOs, validation.

Completion target: PROJECT 2 MVP: Java e-commerce/order backend.

Git/GitHub: Tag v0.1; architecture diagram.

W4D22 — Spring Security

Learn: Authentication, authorization, password hashing, JWT concepts, roles, method security, CORS/CSRF concepts.

Completion target: Add user/admin roles and protected APIs.

Git/GitHub: Security checklist.

W4D23 — Exception Handling + Validation

Learn: Global exception handler, validation annotations, DTO mapping, clean API errors, logging.

Completion target: Production-grade error contract.

Git/GitHub: Add integration tests.

W4D24 — Testing + API Docs

Learn: JUnit, Mockito, Spring Boot tests, integration tests, test profiles; OpenAPI/Swagger concepts.

Completion target: Test critical business flows and document APIs.

Git/GitHub: CI test command.

W4D25 — Docker + Database

Learn: Multi-stage Java image, Compose, MySQL container, environment variables, health checks.

Completion target: Containerize Java app + MySQL.

Git/GitHub: docker-compose.yml.

W4D26 — CI/CD

Learn: GitHub Actions concepts, build/test workflow, secrets, artifact, deployment pipeline, rollback concept.

Completion target: Create CI pipeline and deployment workflow.

Git/GitHub: Badge/status in README.

W4D27 — Monitoring + Observability

Learn: Structured logs, metrics, traces, health checks, Actuator concepts, Prometheus/Grafana concepts, alerts.

Completion target: Add health/metrics and basic dashboards.

Git/GitHub: Document SLO-style targets.

W4D28 — Java Final + Full Stack Integration

Learn: Deploy backend, connect frontend, smoke testing, performance/security review, release process.

Completion target: PROJECT 2 FINAL: Java Spring Boot production API + frontend integration.

Git/GitHub: Tag v1.0 + release notes + portfolio entry.

W5D29 — Python for AI

Learn: Python syntax, virtual environments, pip, functions, classes, files, JSON, requests, typing, exceptions; NumPy/Pandas basics.

Completion target: Create a document/data processing utility.

Git/GitHub: Create ai-roadmap repo with clean structure.

W5D30 — ML Foundations

Learn: Train/validation/test, features/labels, regression/classification, metrics, overfitting, embeddings intuition.

Completion target: Build a small text classification baseline.

Git/GitHub: Notebook + README + requirements.

W5D31 — LLM Fundamentals

Learn: LLM basics, tokens, context windows, transformer intuition, temperature, hallucination, system/user prompts.

Completion target: Build a prompt-driven text assistant.

Git/GitHub: Prompt versions tracked in Git.

W5D32 — LLM APIs

Learn: API clients, structured outputs, streaming, retries, timeouts, cost awareness, secrets management.

Completion target: Build a production-style LLM API service.

Git/GitHub: Backend with .env.example.

W5D33 — Prompt Engineering

Learn: Role/task/context, few-shot, decomposition, constraints, JSON schema, self-checking, prompt injection awareness.

Completion target: Create reusable prompt templates + evaluation set.

Git/GitHub: Version prompts in repo.

W5D34 — LLM App Architecture

Learn: Frontend/backend separation, model gateway, caching concepts, rate limits, conversation state, logging.

Completion target: Build chat application architecture.

Git/GitHub: Architecture diagram + API docs.

W5D35 — AI Mini Project

Learn: Combine Python + LLM API + frontend/API + persistence.

Completion target: PROJECT 3: AI Study/Document Assistant MVP.

Git/GitHub: Tag v0.1; demo video/GIF in README.

W6D36 — Embeddings

Learn: Semantic similarity, embedding vectors, cosine similarity, dense retrieval, metadata.

Completion target: Embed a small document corpus and perform similarity search.

Git/GitHub: Store sample data + evaluation queries.

W6D37 — Vector Databases

Learn: Vector DB concepts, collections, metadata filters, top-k search; Chroma/FAISS concepts.

Completion target: Build vector search service.

Git/GitHub: Benchmark simple retrieval.

W6D38 — RAG Fundamentals

Learn: Ingestion → parsing → chunking → embedding → indexing → retrieval → prompt → generation.

Completion target: Build first RAG pipeline.

Git/GitHub: Document architecture and data flow.

W6D39 — RAG Quality

Learn: Chunk size/overlap, metadata, hybrid retrieval concept, reranking, citations, context compression.

Completion target: Improve Project 3 with citations and better retrieval.

Git/GitHub: Create 20+ evaluation questions.

W6D40 — RAG with LangChain

Learn: Loaders, splitters, retrievers, chains, document stores; abstraction tradeoffs.

Completion target: Rebuild RAG with a framework.

Git/GitHub: Keep a simple implementation for understanding.

W6D41 — RAG API + UI

Learn: FastAPI concepts, Pydantic, async endpoints, streaming, file upload, document lifecycle.

Completion target: Create upload → index → ask workflow.

Git/GitHub: API tests + README.

W6D42 — RAG Final

Learn: Evaluation, failure analysis, security, prompt injection defenses, observability.

Completion target: PROJECT 4: Production-style RAG Knowledge Assistant.

Git/GitHub: Tag v1.0 + evaluation report.

W7D43 — Agent Fundamentals

Learn: Agent vs chatbot, tools, planning, action/observation loops, tool schemas, function calling.

Completion target: Build a tool-using assistant with calculator/search-style tools.

Git/GitHub: Tool interfaces documented.

W7D44 — Tool Calling

Learn: Typed tool inputs, validation, tool errors, retries, permissions, deterministic tools.

Completion target: Add 3–5 safe tools to agent.

Git/GitHub: Unit test every tool.

W7D45 — Agent Workflows

Learn: State machines, sequential/parallel workflows, routing, human-in-the-loop, retries.

Completion target: Build multi-step workflow agent.

Git/GitHub: Draw state/sequence diagram.

W7D46 — LangGraph

Learn: Graph state, nodes, edges, conditional routing, persistence/checkpoints, interrupts.

Completion target: Implement agent workflow using LangGraph.

Git/GitHub: Tag v0.1.

W7D47 — Memory + MCP

Learn: Short/long-term memory concepts, session state, retrieval memory; MCP concepts, servers/tools/resources.

Completion target: Build an agent that can access controlled external tools.

Git/GitHub: Document tool permissions and threat model.

W7D48 — Agent Security

Learn: Prompt injection, tool abuse, data exfiltration, least privilege, allowlists, output validation, sandboxing concepts.

Completion target: Add guardrails and approval for risky actions.

Git/GitHub: SECURITY.md + adversarial tests.

W7D49 — Agent Project MVP

Learn: Integrate tools, memory, RAG, structured outputs, workflow orchestration.

Completion target: PROJECT 5: Agentic Research/Task Automation Assistant.

Git/GitHub: Tag v0.9 + demo.

W8D50 — Agent Evaluation

Learn: Golden datasets, task success, groundedness, tool correctness, latency, cost, regression tests.

Completion target: Create 30–50 evaluation cases.

Git/GitHub: Automated eval command.

W8D51 — AI Observability

Learn: Tracing concepts, token/cost tracking, latency, errors, tool traces, dashboards, alerts.

Completion target: Instrument agent and RAG pipeline.

Git/GitHub: Observability README.

W8D52 — Production API

Learn: FastAPI/Spring integration options, auth, rate limiting, background jobs, queues concepts, webhooks.

Completion target: Expose production API around agent.

Git/GitHub: OpenAPI docs + tests.

W8D53 — Docker + CI/CD for AI

Learn: Containerization, environment secrets, model/API configuration, CI tests, deployment strategy.

Completion target: Containerize AI system and automate tests/build.

Git/GitHub: GitHub Actions workflow.

W8D54 — Cloud Deployment

Learn: Deploy frontend/backend/vector store; environment configuration; health checks; rollback.

Completion target: Deploy PROJECT 5.

Git/GitHub: Production URL + release notes.

W8D55 — End-to-End Hardening

Learn: Security review, cost controls, rate limits, prompt injection tests, failure handling, backup/recovery concepts.

Completion target: Fix top 10 issues from review.

Git/GitHub: v1.0 release candidate.

W8D56 — Capstone + Portfolio

Learn: Architecture explanation, system design, demo, metrics, tradeoffs, interview questions, GitHub hygiene.

Completion target: PROJECT 5 FINAL + portfolio: MERN app + Java app + RAG + Agentic AI.

Git/GitHub: Final v1.0 release, pinned repos, resume bullets, demo video, retrospective.

Final Project Portfolio

PROJECT 1 — MERN Production App

Stack: React + Node + Express + MongoDB + JWT + tests + Docker + CI/CD + deployment + monitoring

A complete task/product/e-commerce-style application with authentication, roles, search, pagination and production deployment.

PROJECT 2 — Java Spring Boot Backend

Stack: Java + Spring Boot + Spring Security + JPA + MySQL + JUnit + Docker + CI/CD + Actuator

Production-style e-commerce/order API with role-based access, validation, tests and monitoring.

PROJECT 3 — AI Study/Document Assistant

Stack: Python + LLM API + FastAPI + React + structured output + persistence

Upload or enter content and get structured explanations, summaries and study outputs.

PROJECT 4 — RAG Knowledge Assistant

Stack: Python + embeddings + vector DB + RAG + citations + evaluation + API

Ask questions over your own documents with source-aware answers.

PROJECT 5 — Agentic AI Capstone

Stack: LLM + tools + RAG + LangGraph + memory + MCP concepts + guardrails + observability + deployment

A controlled research/task automation agent that plans work, calls tools, retrieves knowledge, records state and produces traceable results.

Technology Checklist

Area	Technologies / Skills
Frontend	HTML5, CSS3, JavaScript, TypeScript basics, React, React Router, state management concepts, forms, responsive UI
MERN Backend	Node.js, Express, REST, middleware, JWT, validation, MongoDB, Mongoose, security, testing
Java	Java core, OOP, Collections, Streams, Maven, JDBC, SQL/MySQL, Spring Boot, JPA, Spring Security, JUnit/Mockito
DevOps	Linux/CLI basics, Git/GitHub, Docker, Compose, GitHub Actions, environment/secrets, deployment, logs, health checks
Monitoring	Structured logging, metrics, traces, Actuator, Prometheus/Grafana concepts, uptime/error monitoring, alerting
AI/LLM	Python, APIs, tokens, transformers intuition, prompting, structured outputs, streaming, evaluation, cost/latency
RAG	Embeddings, cosine similarity, chunking, vector DB, metadata, retrieval, reranking, citations, evaluation
Agentic AI	Tool calling, workflows, state, LangGraph, memory, MCP concepts, guardrails, human approval, security

Daily Completion Checklist

- Concepts understood — explain them without notes.
- Code written — not just watched/read.
- Project feature completed — visible in the application.
- Tests added for important behavior.
- Git commit(s) made with meaningful messages.
- README updated when architecture/setup/API changes.
- At least one debugging/problem-solving note recorded.
- End-of-day 10-minute recap: what learned, what broke, what to fix tomorrow.

Definition of Done at Day 56

- At least 5 substantial projects are on GitHub.
- At least 2 production-style full-stack applications are deployed.
- At least 1 RAG system and 1 agentic AI system are deployed.
- Every major repository has README, architecture diagram, setup instructions, environment example, tests and release/tag.

- You can explain request flow, authentication, database design, deployment, logs/metrics, RAG pipeline and agent tool-calling flow in an interview.
- You can clone your own repositories on a fresh machine, configure them, run tests, build Docker images and deploy them.